

Working of CoAP and MQTT Protocols in IoT

**Deep Dive: Architectures, Step-by-Step
Mechanics, and Applied Examples**

Presented by Dr. Anand D. Mane

The Need for Specialized IoT Protocols

- Standard HTTP carries too much overhead for resource-constrained, battery-powered IoT nodes.
- Two application-layer protocols dominate the IoT landscape to minimize overhead:
 - CoAP: Two-party, RESTful request/response over connectionless UDP.
 - MQTT: Broker-mediated publish/subscribe over reliable TCP.

CoAP Paradigm

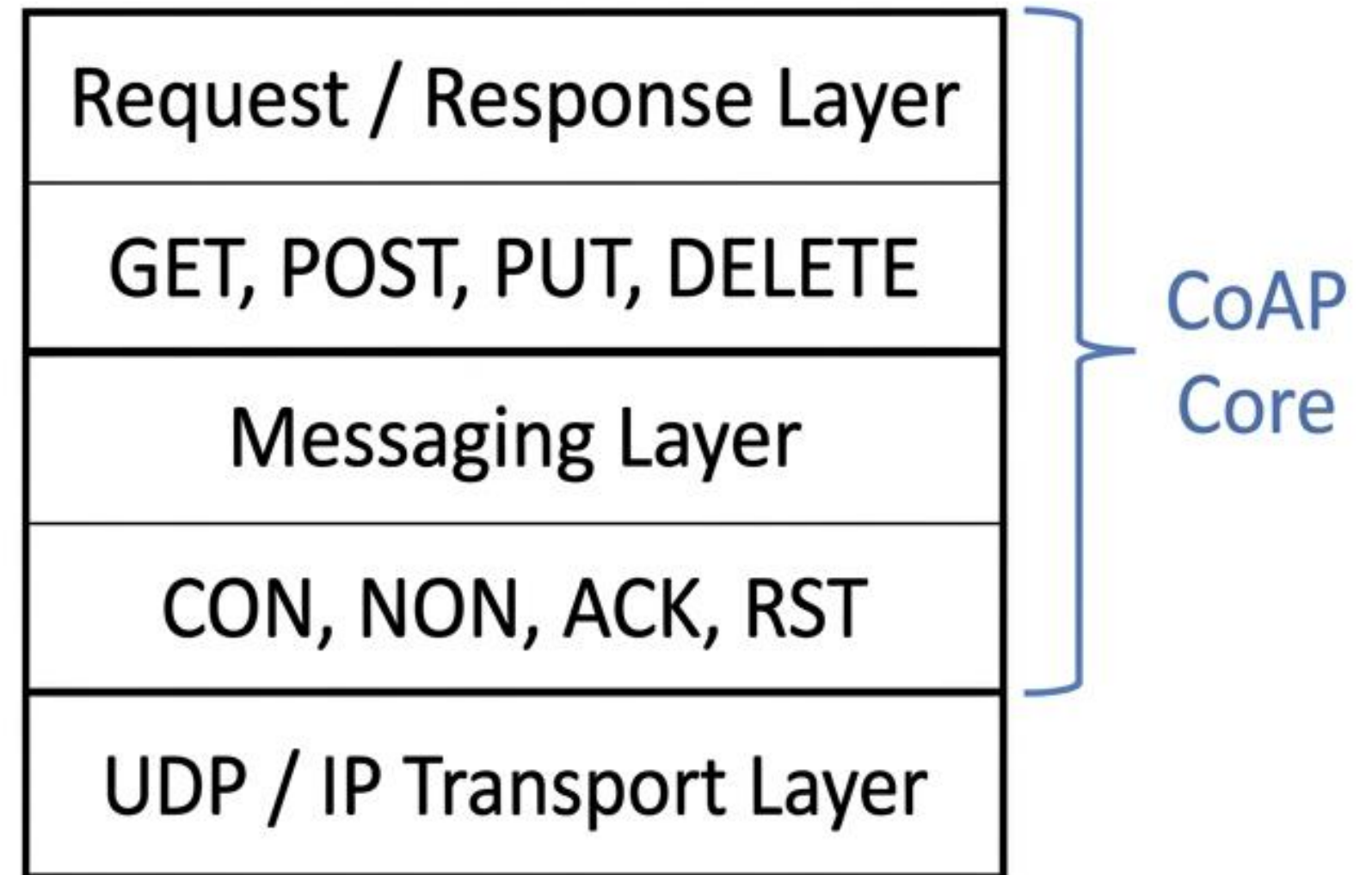


MQTT Paradigm



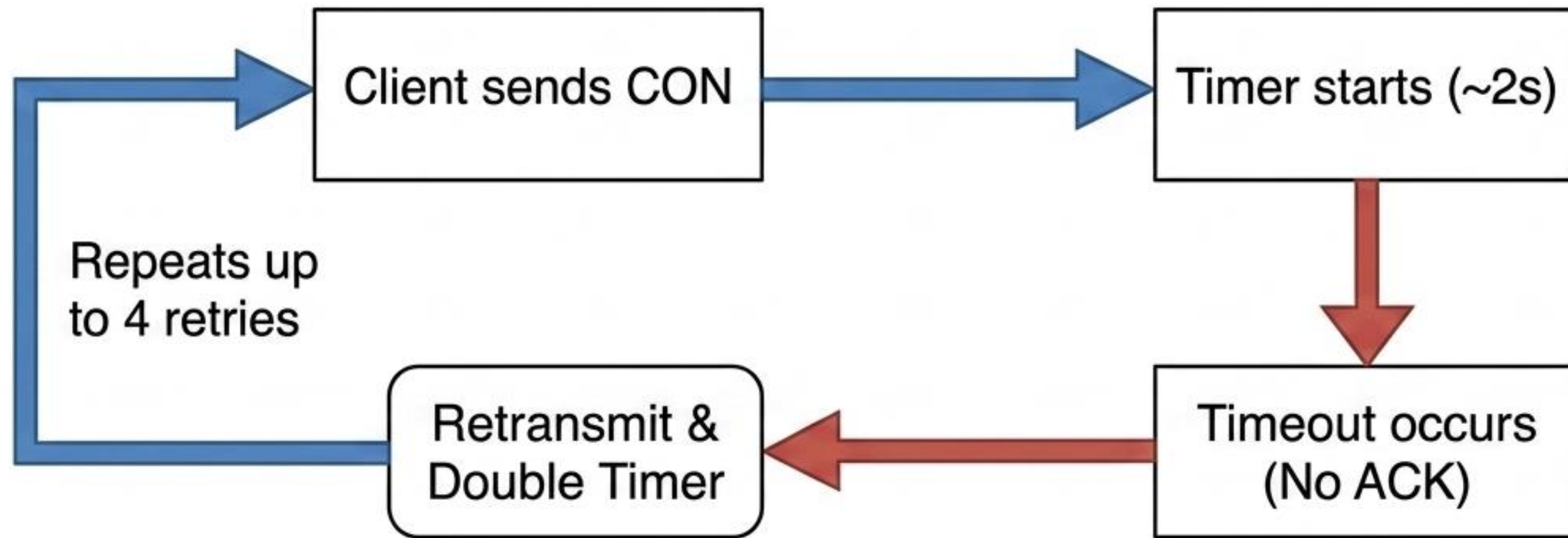
CoAP Architecture Overview

- Runs over connectionless UDP (Port 5683 / 5684 for DTLS).
- Two-layer internal design maps HTTP REST semantics to constrained networks.
- Ultra-low overhead: Compact 4-byte binary header.



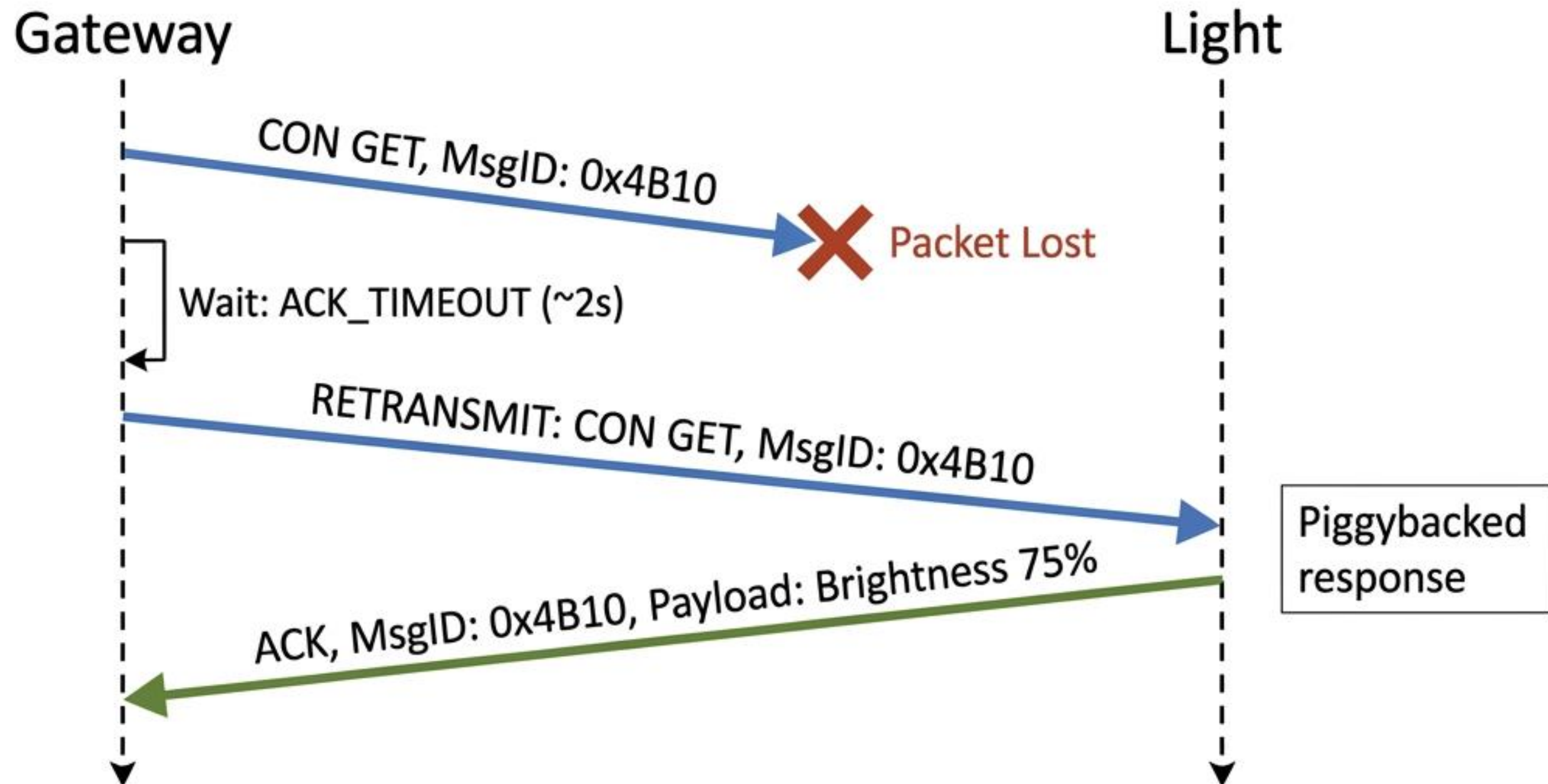
CoAP Messaging Mechanics & Reliability

Step 1: Client constructs request (Method, Msg ID, Token).	Step 2: Select type: CON (Confirmable) or NON (Non-confirmable).	Step 3: Transmit single UDP datagram.	Step 4: If CON, wait for ACK. If lost, trigger Exponential Backoff.
--	--	---------------------------------------	---



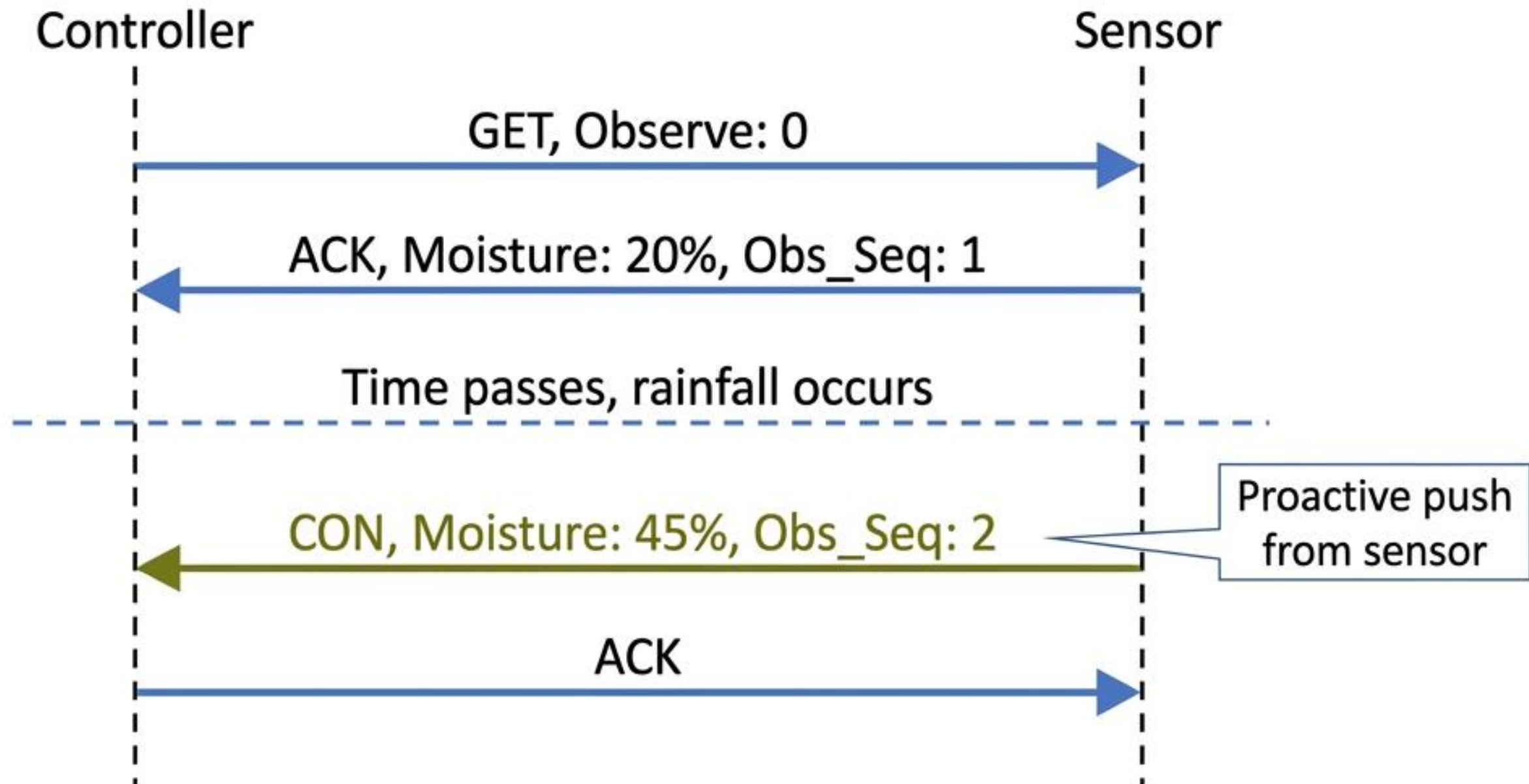
CoAP Example 1: Confirmable GET over Lossy Link

- Scenario: Traffic gateway reads solar street-light brightness over a lossy bridge.
- Message loss is resolved locally via Message IDs without TCP overhead.



CoAP Example 2: Observe Option (Push Updates)

- Scenario: Farm irrigation controller monitors a buried soil-moisture sensor.
- Eliminates continuous polling, preserving the sensor node's battery life.

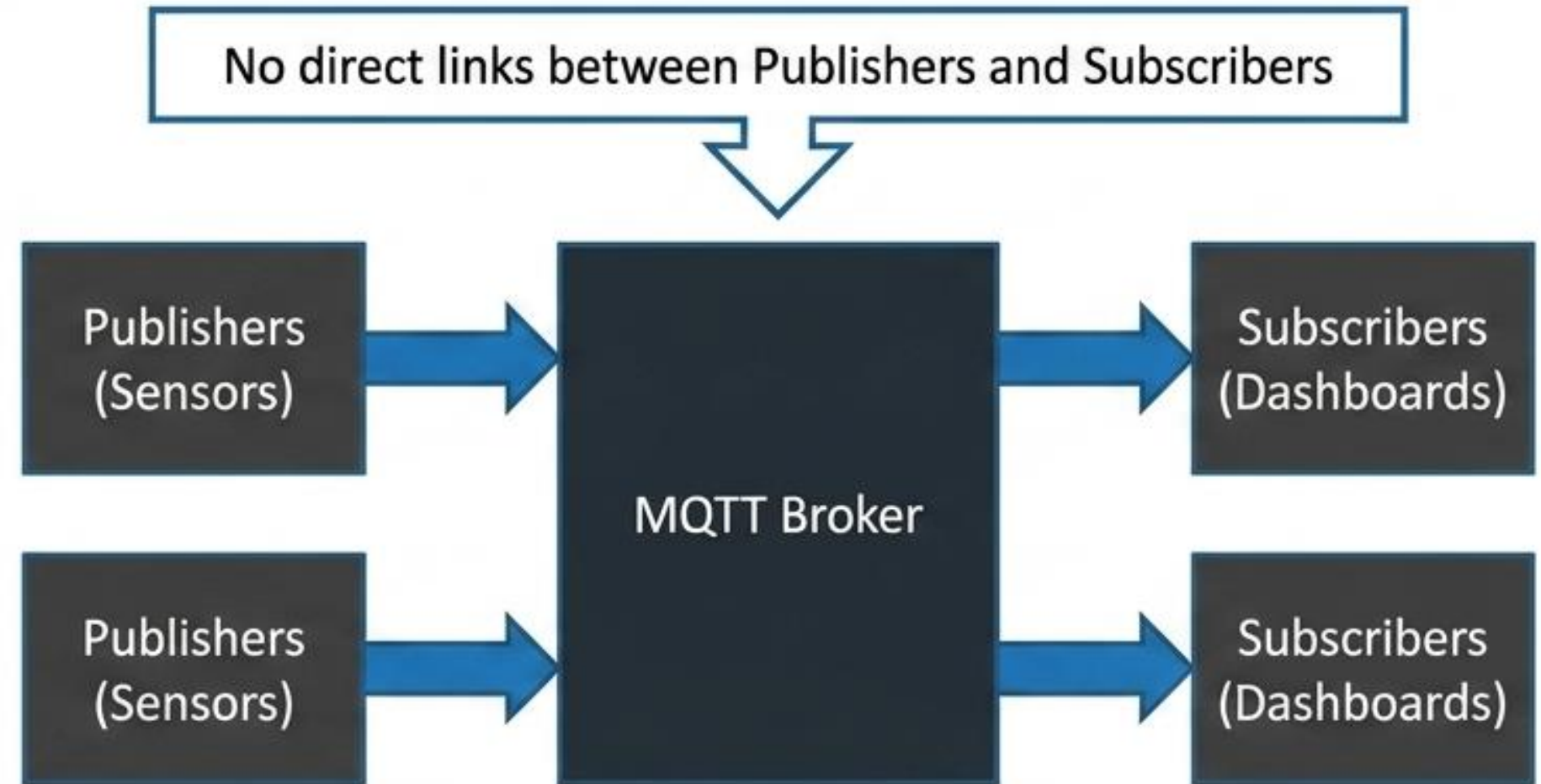


Technical Evaluation: CoAP

Advantages	Disadvantages
• Ultra-low overhead (4–20 bytes header). • No TCP connection setup cost (critical for battery/lossy links). • RESTful design translates natively to HTTP. • Tunable reliability (CON vs NON) and lightweight push (Observe).	• UDP requires custom reliability implementation (deduplication). • Strict two-party protocol; lacks native multi-subscriber fan-out. • DTLS security adds complexity to constrained stacks. • UDP NAT traversal is difficult in standard network setups.

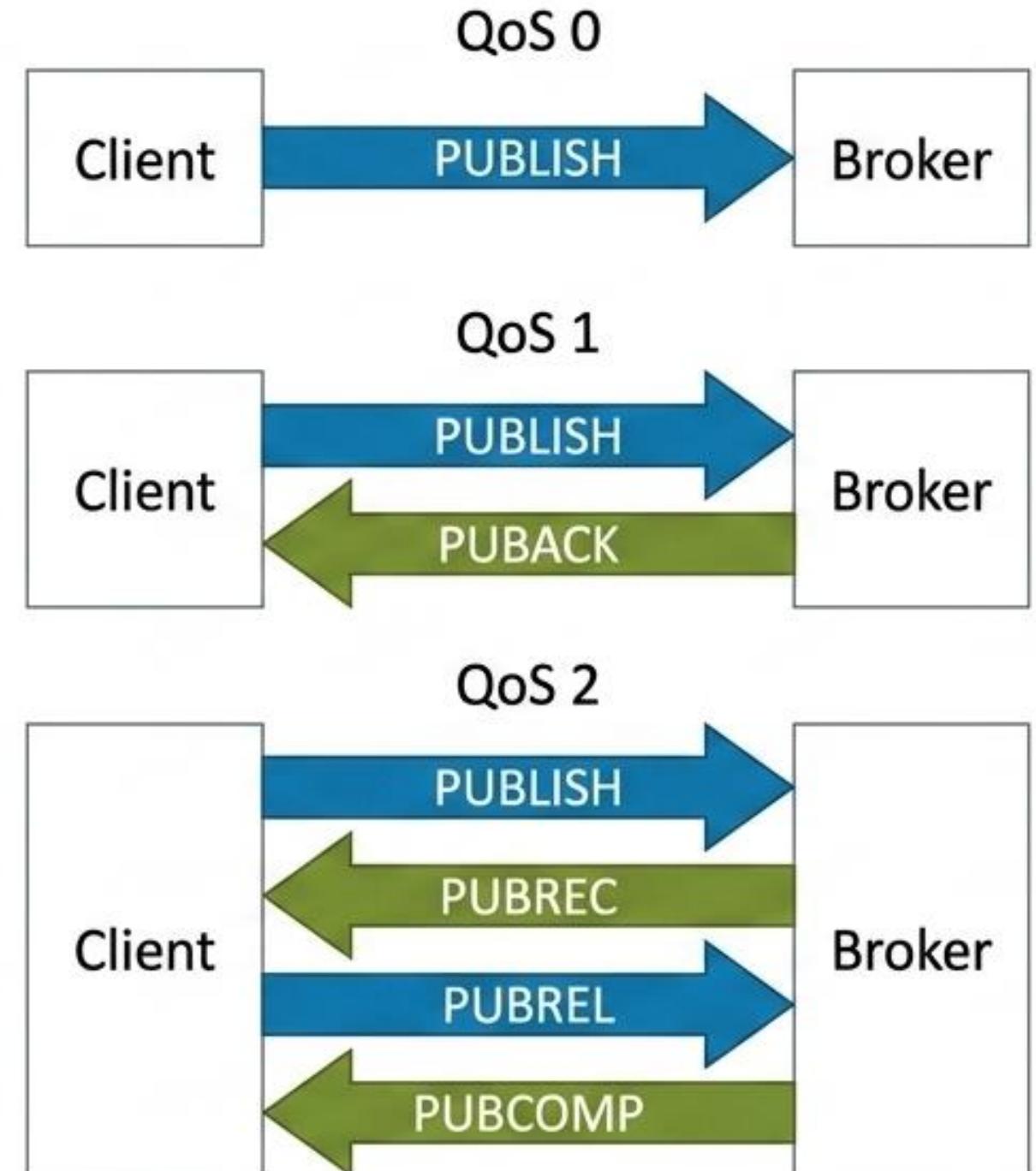
MQTT Architecture Overview

- Runs over reliable TCP (Port 1883 / 8883 for TLS).
- Publish/Subscribe model strictly mediated by a central Broker.
- Publishers and Subscribers are fully decoupled; they never communicate directly.
- Hierarchical routing via Topics with wildcards (+ single, # multi-level).



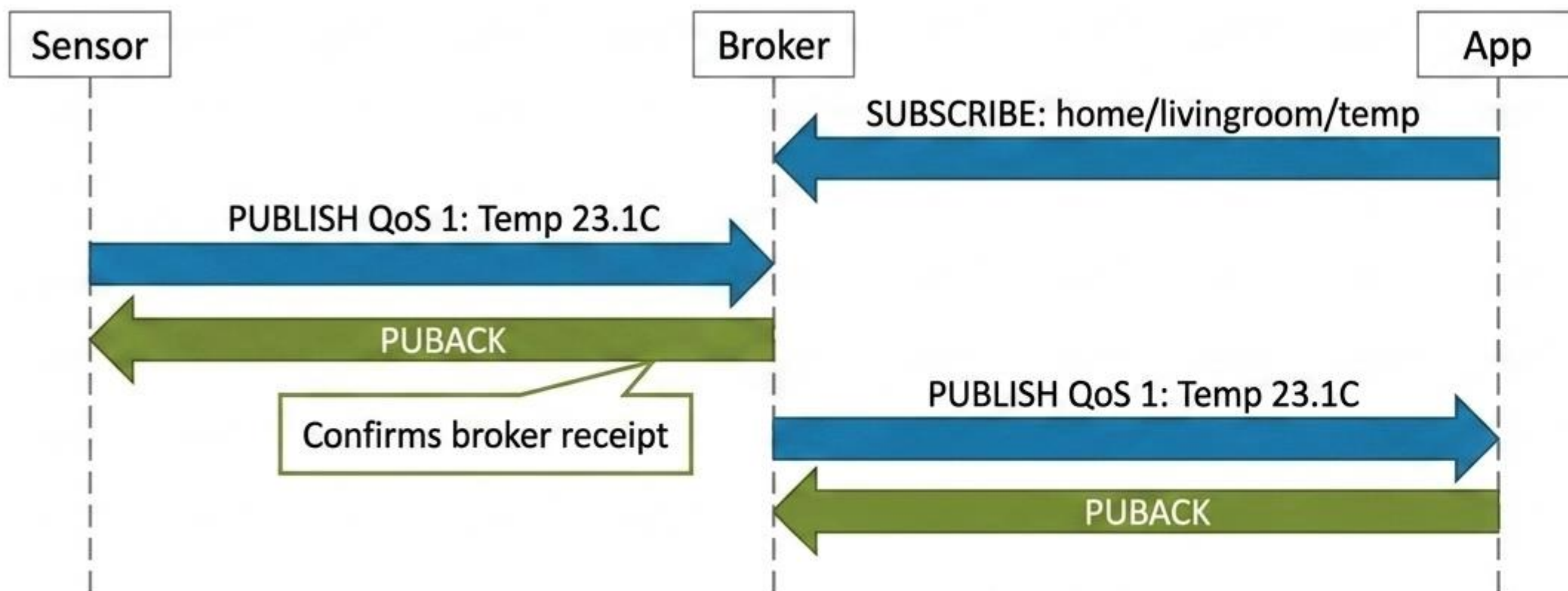
MQTT Quality of Service & State Management

- QoS 0: At most once (Fire and forget).
- QoS 1: At least once (PUBLISH → PUBACK).
- QoS 2: Exactly once (Robust 4-step handshake).
- Retained Messages: Broker stores the last value for immediate delivery to new subscribers.
- Last Will & Testament (LWT): Automatic alert published if a client drops ungracefully.



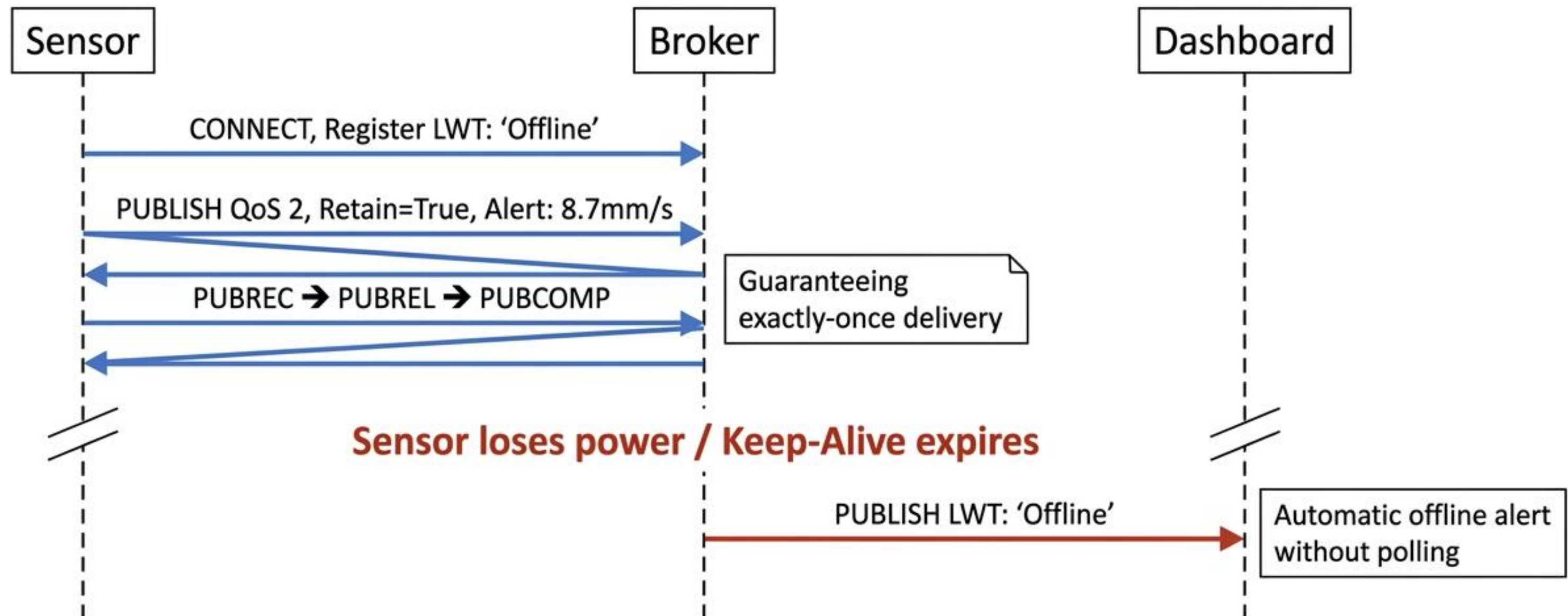
MQTT Example 1: Real-Time Pub/Sub (QoS 1)

- Scenario: Smart-home living room sensor publishes temperature; mobile app subscribes.
 - QoS 1 (“at least once”) ensures the reading is not lost during brief network hiccups.



MQTT Example 2: QoS 2, Retained & LWT

- **Scenario:** Factory vibration monitoring requiring guaranteed alerts (QoS 2) and instant offline detection (LWT).



Technical Evaluation: MQTT

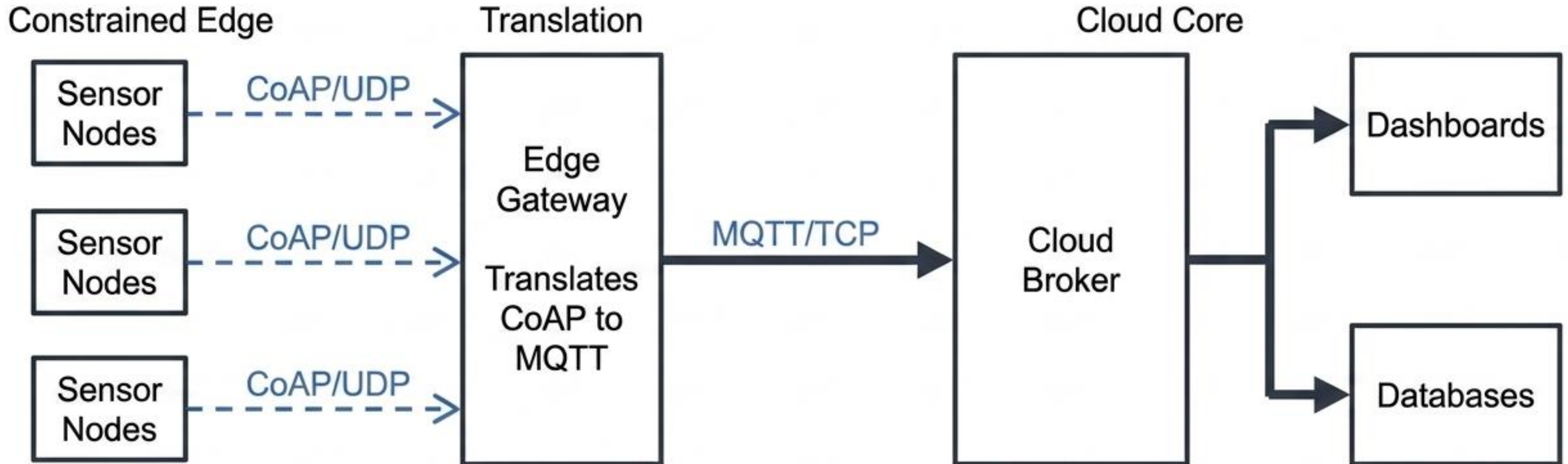
Advantages	Disadvantages
• Broker naturally supports many-to-many, fan-out routing at massive scale.	• Broker acts as a single point of failure (requires complex clustering).
• Tunable QoS (0, 1, 2) lets each message trade reliability against overhead.	• TCP connection and keep-alive overhead drains edge node batteries.
• Retained state & Last Will enable robust offline/liveness detection.	• QoS 2 adds significant latency and protocol complexity.
• TCP foundation bypasses enterprise firewalls easily (Port 443/8883).	• Lacks native request/response (RPC) semantics.

Synthesis: CoAP vs MQTT Diagnostic Matrix

Feature	CoAP	MQTT
Transport Layer	UDP (Connectionless)	TCP (Reliable)
Architecture Paradigm	Client/Server (2-party REST)	Publish/Subscribe (Broker hub)
Reliability Mechanism	Built-in Retries / Message IDs	Configurable QoS Levels (0, 1, 2)
State Management	Observe Option (Push)	Retained Messages & Last Will
Optimal Use Case	Severely constrained, battery-powered edge nodes	Large-scale, asynchronous telemetry networks

Real-World Implementation: Combined Architecture

- Modern IoT architectures do not choose just one protocol; they map them to specific network constraints.
 - CoAP operates at the deeply constrained edge.
 - MQTT aggregates and distributes across the cloud core.



Architectural Summary

- CoAP: The protocol of choice for individually addressable, severely constrained devices requiring RESTful interactions over lossy, low-power links.
 - MQTT: The protocol of choice for large-scale telemetry, requiring asynchronous, robust fan-out messaging across multiple consumers.
-
- **Engineering Conclusion: Protocol selection fundamentally dictates a system's reliability ceiling, edge battery life, and overall scalability. Architect accordingly.**